vehicles. Much of its recent progress has come from classical computing, including large multimodal robot policies and world models trained partly on simulated or synthetic data. Quantum computing, meanwhile, has spent years on a more basic question: how to build processors whose fragile quantum states survive long enough to compute anything useful.

The two communities meet mainly at optimization. Robots constantly face choices about ordering, assignment and configuration, and optimization happens to be one of the areas where quantum algorithms are studied most actively. That overlap is real, but it is narrower than what the headlines tend to suggest.

What a quantum processor actually does

A quantum processing unit, or QPU, is a co-processor. It does not replace the CPU or GPU. A classical computer prepares the problem, sends a carefully designed sequence of operations to the QPU, and reads back measurement results. Those operations act on qubits, and three ideas explain why that matters.

Superposition means a qubit's state is described by amplitudes over 0 and 1 rather than by one definite value, and a register of many qubits carries amplitudes over an enormous number of bit strings at once. That sounds like parallel search, and it is often described that way, but the description misleads. A measurement returns a single bit string, sampled according to those amplitudes. A quantum computer does not try every answer and politely hand back the best one.

The useful part comes from interference. Amplitudes behave somewhat like waves: they can add up or cancel out. A well-designed algorithm arranges its operations so that routes towards poor answers tend to cancel while routes towards good answers reinforce, raising the odds of measuring something worthwhile. Entanglement links qubits so their joint state cannot be described one qubit at a time, giving algorithms correlations ordinary bits cannot reproduce directly. None of this guarantees a speedup. Advantage depends on a problem's structure and an algorithm that exploits it; there is no universal speedup.

Physical qubits, logical qubits and the machine room

The qubits on a chip are physical qubits, and they are noisy: heat, stray fields and imperfect control pulses all corrupt their states. Error correction spreads one logical qubit across many physical ones and repeatedly checks for errors without destroying the encoded information. The catch is that adding qubits only helps when each is good enough. Below a certain error threshold, larger codes protect information better; above it, they make things worse.

In a paper published in *Nature* in December 2024, Google Quantum AI and collaborators reported superconducting quantum memories operating below that threshold, with logical errors falling as the code grew. That is a meaningful step for reliability. It is not a demonstration of general fault-tolerant computing, and it says nothing about robotics performance.

Practical infrastructure also shapes what a QPU can offer a robot. Superconducting processors sit inside dilution refrigerators close to absolute zero, wrapped in racks of classical control electronics, and most users reach them over a network through cloud services and job queues. That arrangement suits offline studies and batch planning far better than anything that must answer within a control cycle.

Three ways to attack an optimization problem

When people talk about quantum optimization, they usually mean one of three quite different things, and blurring them causes a good deal of confusion.

Gate-based machines run circuits of programmable quantum logic operations. Their best-known optimization method, the Quantum Approximate Optimization Algorithm (QAOA), alternates a layer that encodes the cost function with a layer that mixes candidate solutions, while a classical optimizer tunes the settings between runs. It is a hybrid loop by design, and noisy hardware limits how deep, and therefore how capable, its circuits can be.

Quantum annealers take another route. These specialised devices start in an easy-to-prepare state and gradually reshape an energy landscape so the system ideally settles into a low-energy configuration matching a good solution. They can hold many qubits, but each connects to only a few neighbours, so densely connected problems must be embedded by chaining several physical qubits into one variable. That overhead eats capacity quickly.

Then there are quantum-inspired classical methods: algorithms that borrow ideas from physics, such as annealing-style heuristics and tensor-network techniques, but run on conventional processors. They are often the toughest competition. Any serious claim of quantum benefit must beat them, not merely a naive baseline.

How embodied systems think with their bodies

An embodied system lives inside a feedback loop. It senses through cameras, lidar, joint encoders and force sensors, estimates its own state and the world's, plans what to do, and acts through motors. Then it senses again, because the world never quite matches the plan. For a robot that balances or manipulates objects, the lowest layer of this loop may run hundreds or thousands of times per second, with slower planning layers stacked above it.

Geometry versus dynamics

Two kinds of physics run through that loop. The first is spatial geometry: where things are, how shapes overlap, whether